TECHNICAL REPORT:
QUORIDOR GAME IMPLEMENTATION

CSE472s: Artificial Intelligence
Spring 2026

Provided to:
Dr. Manal Mourad Zaki
Eng. Robeir Samir George

Student Name	ID
Youssef Yacoub	2200649
Mohamed Khaled	2201057
Mousa Abdulaziz	2200257
Jomana Abdelmohsen	2300027
Youssef Ahmed	2301303
Ruba Mohamed	2300551

GitHub Repository Link Demo Video Link

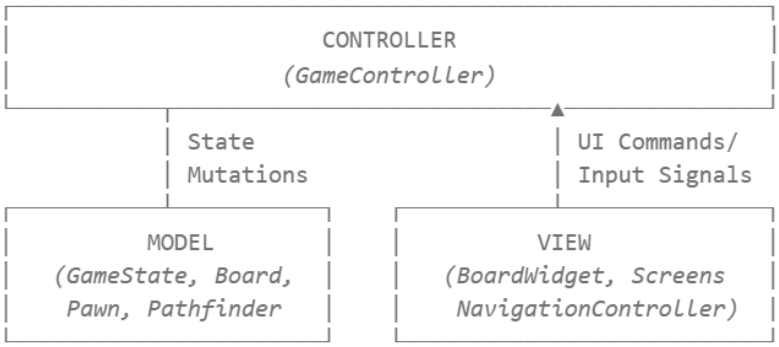
1. Executive Summary & Introduction

This report documents the design, architectural decisions, implementation challenges, and AI algorithms for desktop implementation of the abstract board game **Quoridor**. Developed using **Python** and the **PySide6** (Qt for Python) graphical framework, the project delivers a highly responsive, visually polished single-player (vs. AI) and local multiplayer (pass-and-play) experience.
The primary engineering goals of this project were:

1. **Model-View-Controller (MVC) Separation:** Ensuring that core game logic, coordinate systems, and AI processing remain completely separated from the UI rendering layer.
2. **Mathematical Correctness of Rules:** Guaranteeing spatial operations, such as legal pawn steps, wall-blocking path validations, orthogonal jumps, and diagonal sidesteps, conform precisely to the official game regulations.
3. **Asynchronous Desktop Performance:** Decoupling computationally demanding tasks—such as Minimax search trees—from the main Qt GUI thread to prevent frame drops or UI freezing.
4. **Implementation of Bonus Strategic Features:** Offering robust command-history tracking (**Undo/Redo** command pattern) and serialization pipelines (**saving/loading** game states from JSON format).

2. Design Decisions and Architecture

The system utilizes a structured, decoupled **Model-View-Controller (MVC)** architectural pattern to separate state management, visual presentation, and user interaction.



2.1 The Model Layer (State & Domain Logic)

The model represents the state of the domain without any knowledge of fonts, pixels, color schemes, or window sizes:

- **`Pawn`**: A mutable domain entity representing a player's pawn, defining starting coordinates (row, col) and goal row criteria.
- **`Board`**: Encapsulates the physical spatial configurations of the 9×9 grid.
  It stores placed walls in specialized high-performance sets (*`h_walls`* and *`v_walls`*) and resolves edge-by-edge blockages.
- **`Pathfinder`**: A stateless *BFS* (Breadth-First Search) engine optimizing path validation and depth computations. It communicates exclusively with **`Board`** and **`Pawn`** objects.
- **`GameState`** **(Aggregate Root)**: Orchestrates the structural entities (**`Board`**, list of **`Pawn`** objects, *`PlayerInfo`* structures) and manages turn states, historical action stacks (*`_history`* and *`_redo_stack`*), and validation logic.

2.2 The View Layer (Presentation & Custom Painting)

The view consists of native PySide6 UI screens structured cleanly inside a *`QStackedWidget`* managed by a central **`NavigationController`**:

- **`BoardWidget`**: A custom-painted sub-component using *`QPainter`*. It manages coordinate-scale transformations, mouse tracking, wall placement previews, and interpolated linear transitions for smooth pawn movements.
- **Bento-Grid UI Design**: Adopted throughout the game screens, particularly in the **`RulesScreen`**, optimizing readability across different screen sizes.
- **Decoupled View Routing**: Views communicate changes to controllers entirely via *Qt* Signals, preventing circular dependencies.

2.3 The Controller Layer (Coordination & Asynchrony)

The controller layer translates user events into domain actions and coordinates the feedback loop:

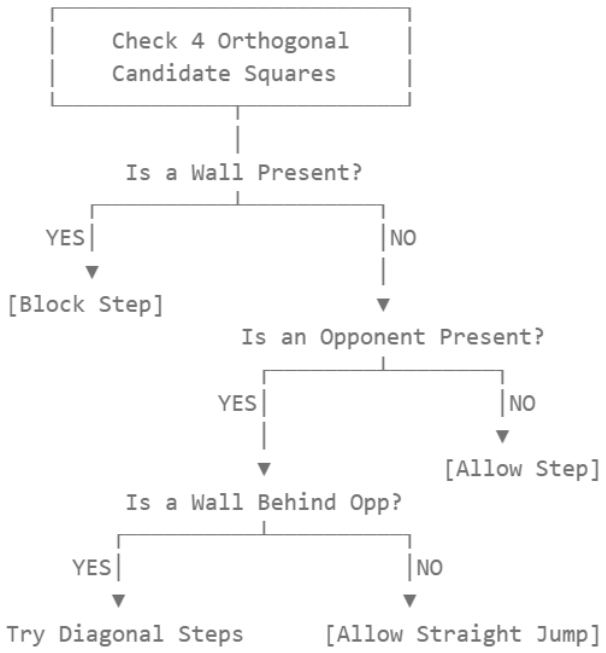
- **`GameController`**: Serves as the central state machine. It handles user action inputs, mutates the aggregate **`GameState`**, and updates the view using a unidirectional data flow through *`_push_display()`*.
- **`NavigationController`**: Manages view transitions without letting screens directly reference or import each other.
- **`AIWorker`**: Subclasses *`QRunnable`* to compute decision paths on a background thread (*`QThreadPool`*), preserving UI responsiveness.

3. Game Rules & Mechanics Verification

To guarantee complete rule of correctness under all board conditions, the model layer implements strict spatial validation routines.

3.1 Orthogonal Step & Jump Logic

The standard movement sequence allows a pawn to travel to any orthogonally adjacent square if no wall exists between the coordinates. The validation sequence in `Board.legal_pawn_moves` is implemented as follows:



1. **Orthogonal Jump**: If an opponent occupies an adjacent square (e.g., North), the engine evaluates the coordinate one step further North.
   If it is in bounds and unblocked by walls, the active player can perform a straight jump.
2. **Diagonal Sidestep**: If a straight jump is blocked by a wall or the board border, the engine checks for perpendicular paths (East and West relative to the opponent's position).
   The pawn can move diagonally to these squares, provided no walls block the lateral transition.

3.2 Wall Coordinates & Overlap Prevention

Walls sit in the gaps between the cells. We represent the 8×8 intersections on the board using a coordinate system where each wall spans exactly two cell-widths:

- A **Horizontal wall** at (r, c) blocks movement between rows r and $r + 1$ for columns and $c + 1$.
- A **Vertical wall** at (r, c) blocks movement between columns c and $c + 1$ for rows r and $r + 1$.

To prevent structural invalidity, `Board` rejects proposed placements that violate any of these conditions:

1. **Duplicate Placement**: A wall already exists at identical coordinates and orientation.
2. **Partial Overlap**: A parallel wall overlaps by starting one unit left/right (for horizontal) or one unit up/down (for vertical).
3. **Cross-Intersections**: A vertical wall cannot cross an existing horizontal wall sharing the exact same intersection coordinate , and vice versa.

4. Path-Finding Validation

A key rule of Quoridor:

"*no wall placement may completely block any player's path to their goal row*".

The game must always preserve at least **one** valid path for each player.

4.1 Path Validation Algorithm

Before any wall is permanently placed, the **`Board`** performs a tentative placement and evaluates player paths using the **`Pathfinder`** utility:

$$\forall p \in Players, Pathfinder.has\_path(board, p.row, p.col, p.goal\_row) == True$$

`Pathfinder` uses a **Breadth-First Search (BFS)** algorithm to find paths on the unweighted 9×9 grid:

```python
@staticmethod  8+ usages  youssef631
def shortest_path_length(
    board: "Board", start_row: int, start_col: int, goal_row: int
) -> int:
    """..."""
    if start_row == goal_row:
        return 0
    cached = _bfs_get(board, start_row, start_col, goal_row)
    if cached is not None:
        return cached

    visited: set[tuple[int, int]] = {(start_row, start_col)}
    # BFS queue stores (row, col, distance)
    queue: deque[tuple[int, int, int]] = deque([(start_row, start_col, 0)])

    while queue:
        row, col, dist = queue.popleft()

        for nrow, ncol in Pathfinder._orthogonal_neighbours(board, row, col):
            if (nrow, ncol) not in visited:
                visited.add((nrow, ncol))
                if nrow == goal_row:
                    _bfs_set(board, start_row, start_col, goal_row, dist + 1)
                    return dist + 1
                queue.append((nrow, ncol, dist + 1))

    _bfs_set(board, start_row, start_col, goal_row, -1)
    return -1  # unreachable
```

If BFS evaluates **-1** for either player, the path is blocked, and the controller rejects the wall placement.

4.2 BFS Performance Optimizations

Running a path-finding search on a 9×9 grid is computationally cheap:

$$O(V + E) = O(81 + 144)$$

However, during deep AI lookaheads where Minimax evaluates thousands of hypothetical states, running path-finding operations repeatedly can impact performance. To resolve this, we implemented two optimizations:

1. **Global BFS Cache**: States are cached using a composite key: *(frozenset(h_walls), frozenset(v_walls), start_row, start_col, goal_row)*.
2. **LRU Eviction Policy**: To keep memory usage stable, the cache clears half of its records when it reaches a limit of 8,192 entries, maintaining stable memory performance during long game sessions.

5. AI Algorithms Explanation

The game includes three distinct AI difficulty tiers to provide an engaging experience for players of different skill levels.

5.1 Static Heuristic Evaluation Function

The core heuristic function used in our AI algorithms evaluates the relative board advantage of the computer opponent. Higher values favor the AI player:

$$S = \left(D_{human} - D_{AI}\right) + \gamma \cdot \left(W_{AI} - W_{human}\right) + 0.1 \cdot M_{AI}$$

Where:

- D_{AI} and D_{human} are the shortest path distances to their respective goal lines, computed via *Pathfinder.shortest_path_length*.
- W_{AI} and W_{human} are the number of remaining walls.
- $\gamma = 0.5$ is a weighting factor that balances wall inventory with path progress.
- M_{AI} is the mobility factor, representing the number of orthogonally accessible neighboring squares from the AI's current coordinate.
  This encourages center-board control and helps avoid corner traps.

The function returns $+\infty$ if the AI reaches its goal line and $-\infty$ if the human player reaches their goal.

5.2 AI Difficulty Levels

1. Novice (Easy Difficulty)

- **Strategy**: A causal agent designed for beginners.
- **Algorithm**:
    - Generates all legal pawn moves.
    - Has a 20% chance to select a random wall from a pre-sorted candidate pool if it still has walls available.
    - If moving the pawn, it evaluates the short-term impact of each move and chooses the step that minimizes its immediate BFS distance to the goal line.

2. Adept (Medium Difficulty)

- **Strategy**: A competitive agent that makes smart, reactive decisions.
- **Algorithm**:
    - Use a **Greedy 1-Ply Search** to evaluate all immediate legal moves and wall placements.
    - Calculates the static heuristic value for every possible next state.
- **Oscillation Penalty**: To prevent the AI from getting stuck in repetitive back-and-forth loops, the evaluation function applies a heavy penalty (-10.0 points) if a move returns the pawn to its previous position.
- **Wall Optimization**: Only places a wall if it increases the opponent's shortest path by at least 2 steps, preventing the AI from wasting walls on minor disruptions.

3. Architect (Hard Difficulty)

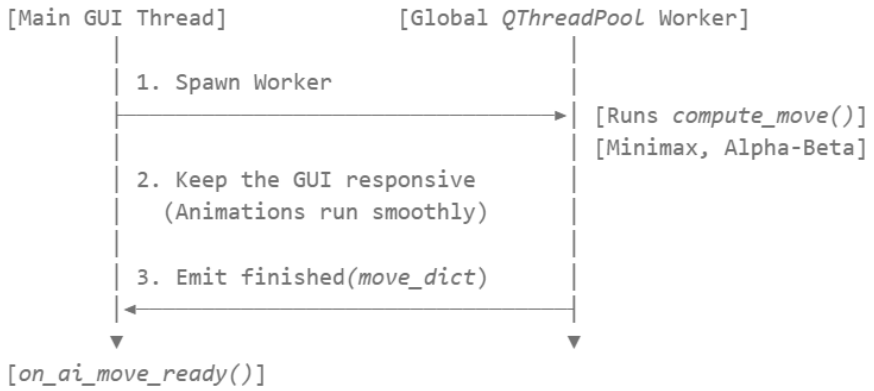
- **Strategy**: A highly strategic agent designed to challenge experienced players.
- **Algorithm**:
    - Uses **Minimax Search with Alpha-Beta Pruning** to anticipate opponent strategies.
    - **Dynamic Search Depth**: The lookahead depth scales dynamically based on the number of walls remaining on the board:
        - $> 4\ walls\ left$: Depth = 3 plies.
        - $3\ to\ 4\ walls\ left$: Depth = 4 plies.
        - $\leq 2\ wallsleft$ (Endgame): Depth = 6 plies, enabling deep endgame calculations.

- **Smart Move Ordering**: To prune irrelevant branches as early as possible, wall candidates are pre-sorted in descending order based on how much they block the opponent's path before running the Minimax search.

6. Implementation Challenges and Solutions

6.1 UI Thread Blocking & Qt Event Loop Integration

- **Challenge**: Running deep Minimax searches (such as 6-ply lookaheads in the endgame) on the main GUI thread causes the desktop window to freeze and stop responding to events.
  → **Solution**: We decoupled the AI computation from the main loop using Qt's thread-safe concurrent design pattern:



The **`GameController`** delegates execution to an **`AIWorker`** (which inherits from *`QRunnable`*) and submits it to *`QThreadPool.globalInstance()`*.
The worker uses a deep copy of the game state to ensure thread safety.
Once computation is complete, it communicates back to the main thread using a custom Qt Signal (*`_AIWorkerSignals.finished`*).

6.2 Responsive Custom Canvas Scaling

- **Challenge**: The game board needs to render correctly across various window dimensions without stretching the board or cells out of proportion.
  → **Solution**: We implemented custom scaling inside **`BoardWidget`** (*`resizeEvent`* and *`paintEvent`*).
  The coordinate system uses a base scale factor:

$$scale = (width\,,\ height)\ /\ baseSize$$

This factor is applied to all grid cells, wall gaps, and padding, keeping the board square and centered at any resolution.

6.3 Python 3.10 Syntax Limitations with Nested Dict Lookups in f-strings

Challenge:
Embedding dictionary key lookups inside f-strings (e.g., `f"{COLORS['primary']}"`) can cause syntax parsing errors on Python 3.10 and 3.11, depending on the outer string quotes.
→ **Solution**:
We extracted all required design tokens into module-level variables (such as `_C_PRIMARY` and `_C_OUTLINE_V`) within `views/screens/rules_screen.py`.
This ensures full compatibility across all ≥ 3.10 Python versions.

7. Assumptions Made During Development

1. **Two-Player Limit**: The game is designed specifically for two players.
   Expanding a four-player mode would require changes to starting coordinates, goal conditions, and wall configurations.
2. **No Mid-Turn State Saves**: Saving captures the state at the end of completed turns.
   Mid-turn interactions, such as an active hover preview or temporary pawn selections, are intentionally cleared during serialization.
3. **Standard Screen Resolutions**: We assume minimum system resolution of 1024×768 pixels.

8. References and Resources Used

1. **Official Quoridor Rules**: Mirko Marchesi, Gigamic Games.
   Official Quoridor Rules Reference.
2. **Python documentation**: for project development Python documentation.
3. **PySide6 & Qt Documentation**: "Qt for Python" Reference API Guide.
   Qt Documentation.
4. **Alpha-Beta Pruning Optimization**: Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*. Prentice Hall.